

0. SETUP (1ª CÉLULA)

```
import pandas as pd
postos = pd.read_csv("postos_sp_2025.csv", sep=";",
decimal=",")
distrib = pd.read_csv("distribuicao_municipios_sp_2025.csv",
sep=";", decimal=",")

# Strip espaço fantasma em todas colunas de texto
for col in postos.select_dtypes(include="object").columns:
    postos[col] = postos[col].str.strip()

postos["Mes"] = (postos["Data da Coleta"].str[6:10] + "-"
+ postos["Data da Coleta"].str[3:5])
print(postos.shape, distrib.shape); postos.head()
```

CSV BR: sempre sep=";" + decimal=",". Sem isso, parse quebra ou número vira string.

1. INSPEÇÃO

df.head(n) tail() sample()	linhas
df.shape (sem ())	(lin, col)
df.info() dtypes columns	tipos/nomes
df.describe()	num: count/mean/std/min/Q/max
describe(include="all")	+ texto
len(df)	nº linhas

Pra cada DataFrame novo, sempre roda shape+info()+head() ANTES de qualquer análise. Confere tamanho, tipo das colunas, e amostra visual. info() mostra se algum número virou string (problema do decimal).

2. SELEÇÃO DE COLUMNA

```
df["c"]          # Series (1D)
df[["c1","c2"]]  # DataFrame (várias, ordem da lista)
df[["c"]]        # DataFrame com 1 col (≠ Series)
```

1 colchete vira Series, 2 colchetes viram DataFrame. Pra reordenar colunas, usa 2.

DataFrame é a tabela inteira (2D, várias colunas). Series é uma coluna sozinha (1D). Series tem métodos de coluna (mean, str, value_counts). DataFrame não. 99% das operações tu quer Series (1 colchete).

3. FILTRO LÓGICO (LINHA)

```
cond = df["Produto"] == "GASOLINA"    # Series booleana
df[cond]                               # filtra

df[df["Produto"] == "GASOLINA"]
df[df["Valor de Venda"] > 6]
df[df["Bandeira"] != "BRANCA"]
df[df["Bandeira"].isin(["VIBRA", "RAIZEN"])]
len(df[df["Produto"]=="GASOLINA"])    # quantas
```

Filtro é sempre 2 etapas: cria Series booleana (True/False por linha) com a condição, aplica dentro do df[...]. Pandas vê booleana ali e fica só com linhas True. Ops: == != > < >= <= e .isin([...]) pra "está na lista".

4. AND / OR / NOT

```
# NUNCA and/or/not. Sempre & | ~
df[(df["Produto"]=="GASOLINA") & (df["Valor de Venda"]>6)]
df[(df["Bandeira"]=="VIBRA") | (df["Bandeira"]=="RAIZEN")]
df[~(df["Bandeira"]=="BRANCA")]

# Recomendado (cada cond como variável):
c1 = df["Produto"]=="VIBRA"
c2 = df["Valor de Venda"]>6
df[c1 & c2]
```

Cada condição entre parênteses. Esquecer é erro #1.

Pandas tem operadores próprios pra booleanas: & (E), | (OU), ~ (NÃO). NUNCA usa and/or/not de Python puro. Cada condição entre parênteses, ou a precedência quebra silenciosamente.

5. LOC / ILOC

```
df.loc[cond, "Valor de Venda"]
df.loc[cond, ["Revenda","Valor de Venda"]]
df.loc[df["Produto"]=="GASOLINA","Valor de Venda"].mean()
df.iloc[0]          # 1ª linha (posição)
df.iloc[0:5, 0:3]   # FIM EXCLUSIVO
df.loc[5:10]        # FIM INCLUSIVO
```

	loc	iloc
Usa	rótulo	posição
Fim	inclusivo	exclusivo

Pra filtrar linha E selecionar coluna ao mesmo tempo, OBRIGATÓRIO usar loc. Sem ele, dá warning. Vírgula separa: antes é seletor de linha, depois é seletor de coluna.

6. .STR (STRINGS NA COLUMNA)

```
df["c"].str.upper() / .lower() / .title() / .strip()
df["c"].str.len()          # tamanho
df["c"].str.contains("POSTO") # booleana (filtro!)
df["c"].str.startswith("AUTO") / .endswith("LTDA")
df["c"].str.replace("LTDA","") # SUBSTRING
df["c"].str[0] .str[-3:] .str[6:10] # slice
df["c"].str.split()        # vira lista
df["c"].str.split(expand=True) # vira DF
df["c"].str.split().str[-1]  # último elemento

df["c"].str.upper().str.replace("LTDA","").str.strip() #
encadeia

Sem .str: 'Series' object has no 'upper'. Sempre .str antes.

.str é a ponte entre Series e métodos de string Python. Sem ela,
df["c"].upper() falha porque Series não tem upper. Com .str, o
método é aplicado em cada string da coluna, retorna Series nova.
```

7. REPLACE VS .STR.REPLACE

Forma	Compara
col.replace("SP","BA")	valor inteiro
col.str.replace("SP","BA")	substring no texto

```
df["Estado"] = df["Estado"].replace("SP","Sao Paulo")
df["Bandeira"] =
df["Bandeira"].replace({"BRANCA":"SEM","VIBRA":"BR"})

replace compara o valor INTEIRO (célula tem que ser exatamente "SP").
.str.replace troca SUBSTRING dentro do texto (parcial). Dicionário
{velho: novo} troca vários valores de uma vez.
```

8. CRIAR / ATUALIZAR COLUMNA

```
df["nova"] = df["a"] + df["b"] # cria à direita
df["a"] = df["a"] * 1.1        # substitui
df["Pais"] = "Brasil"          # constante
df["Cid_U"] = df["Município"].str.upper()
```

Pegadinha mortal: df = df["c"].str.lower() DESTRÓI o DF (vira Series). Esquerda tem que ser df["c"], NUNCA df.

Pra criar OU atualizar coluna, esquerda é df["c"]. Se a coluna já existe, substitui; se não, cria à direita da última. Mesma sintaxe pros dois casos.

9. ASTYPE (CONVERTER TIPO)

```
df["c"] = df["c"].astype(float / str / int)

# Se Valor veio string (decimal não pegou):
df["Valor de Venda"] = (df["Valor de Venda"]
.str.replace(",",".").astype(float))

String + número falha. Converte: "C" + df["id"].astype(str).
```

Converte o tipo da coluna inteira. Usa quando pandas leu errado (número virou string) ou quando precisa concatenar/operar tipos diferentes.

10. ESTATÍSTICA NUMÉRICA

```
df["c"].mean() .median() .std() .var()
df["c"].min() .max() .sum() .count() # count = não-
nulos
df["c"].quantile(0.25) .quantile(0.75)
df["c"].describe() # tudo de uma vez
df["c"].idxmax() # ÍNDICE do maior
```

Sem (): df["c"].mean retorna bound method, não o número.

Toda estatística aplicada em UMA coluna específica, retorna 1 número. df["c"].mean(), não df.mean(). idxmax retorna o ÍNDICE (posição/rótulo) onde tá o máximo, não o valor.

17. CASO REAL: POSTOS + DISTRIB (MARGEM)
1. Cria Mes em postos (formato YYYY-MM): postos["Mes"] = (postos["Data da Coleta"].str[6:10] + "-" + postos["Data da Coleta"].str[3:5])
2. Junta pela chave TRIPLA (sem isso, cartesiano): juntos = pd.merge(postos, distrib, on=["Municipio","Produto","Mes"], how="inner")
3. Margem (revenda – atacado): juntos["margem"] = juntos["Valor de Venda"] – juntos["Preco_Distribuicao"]
4. Margem média por bandeira (ranking): juntos.groupby("Bandeira") ["margem"].mean().sort_values(ascending=False)

18. CENÁRIOS TÍPICOS (COLE ADAPTANDO)
Postos distintos em SP capital df[df["Municipio"]=="SAO PAULO"]["CNPJ"].nunique()
Top 5 cidades gasolina mais cara (df[df["Produto"]=="GASOLINA"] .groupby("Municipio")["Valor de Venda"].mean() .sort_values(ascending=False).head(5))
Bandeira mais freq. em SP df[df["Municipio"]=="SAO PAULO"] ["Bandeira"].value_counts().head(1)
Município com diesel mais caro (nome) (df[df["Produto"]=="DIESEL S10"] .groupby("Municipio")["Valor de Venda"].mean().idxmax())
Preço médio mês a mês de gasolina em SP sp = df[(df["Produto"]=="GASOLINA") & (df["Municipio"]=="SAO PAULO")].copy() sp["Mes"] = sp["Data da Coleta"].str[6:10]+"-"+sp["Data da Coleta"].str[3:5] sp.groupby("Mes")["Valor de Venda"].mean()
Revenda contém "AUTO POSTO" E bandeira VIBRA c1 = df["Revenda"].str.contains("AUTO POSTO") c2 = df["Bandeira"]=="VIBRA" df[c1 & c2]
Municípios com gasolina máxima > 7 g = df[df["Produto"]=="GASOLINA"] m = g.groupby("Municipio")["Valor de Venda"].max() m[m > 7]
Distribuição por produto (1 linha) df.groupby("Produto")["Valor de Venda"].describe()
Qtd bandeiras por cidade (top 10) df.groupby("Municipio") ["Bandeira"].nunique().sort_values(ascending=False).head(10)
Mês mais caro pra gasolina g = df[df["Produto"]=="GASOLINA"].copy() g["Mes"] = g["Data da Coleta"].str[6:10]+"-"+g["Data da Coleta"].str[3:5] g.groupby("Mes")["Valor de Venda"].mean().idxmax()
Margem média por município (top 10) juntos.groupby("Municipio") ["margem"].mean().sort_values(ascending=False).head(10)
2 resultados na mesma célula display(df["a"].mean()) display(df["b"].mean())
Qtos produtos distintos vendidos df["Produto"].nunique() df["Produto"].unique() # lista os nomes
Qtos municípios diferentes df["Municipio"].nunique()

Preço médio geral (todos produtos) df["Valor de Venda"].mean()
% de postos sem bandeira (BRANCA) (df["Bandeira"]=="BRANCA").mean() * 100 # ou: df["Bandeira"].value_counts(normalize=True) ["BRANCA"]*100
Diferença média entre revenda e atacado por produto juntos.groupby("Produto")["margem"].mean()
Top 3 bandeiras com mais postos físicos (CNPJs distintos) df.groupby("Bandeira") ["CNPJ"].nunique().sort_values(ascending=False).head(3)
Cidade com maior variância de preço da gasolina (df[df["Produto"]=="GASOLINA"] .groupby("Municipio")["Valor de Venda"].std().idxmax())
Bandeira mais comum vendendo DIESEL df[df["Produto"]=="DIESEL S10"] ["Bandeira"].value_counts().head(1)
Maior preço já registrado (e onde) df["Valor de Venda"].max() # valor df.loc[df["Valor de Venda"].idxmax()] # linha inteira
Postos (CNPJ) que mais aparecem (coletas) df["CNPJ"].value_counts().head(5)
Bandeiras presentes em SP capital df[df["Municipio"]=="SAO PAULO"]["Bandeira"].unique()
Preço médio por bandeira, só gasolina df[df["Produto"]=="GASOLINA"].groupby("Bandeira")["Valor de Venda"].mean().sort_values()
Coletas em janeiro de 2025 postos[postos["Mes"]=="2025-01"] # ou: postos[postos["Data da Coleta"].str.contains("/01/2025")]
Mediana do preço por produto df.groupby("Produto")["Valor de Venda"].median()
Variação de preço (max – min) por bandeira g = df.groupby("Bandeira")["Valor de Venda"] g.max() – g.min()
Posto mais caro de cada município (CNPJ) df.loc[df.groupby("Municipio")["Valor de Venda"].idxmax(), ["Municipio","CNPJ","Revenda","Valor de Venda"]]
Renomear coluna df = df.rename(columns={"Valor de Venda":"Revenda"})
Selecionar/reordenar colunas df = df[["Mes","Municipio","Produto","Valor de Venda"]]